# Deep Implicit Layers for No-Arbitrage Implied Volatility Surface Interpolation*

1st Rafael Zimmer
*Institute of Mathematics and Computer Sciences*
São Paulo, Brazil
rafael.zimmer@usp.br

***Abstract*—We implement a neural network architecture based on deep implicit layers with the objective of optimizing the parameters of multiple SABR (stochastic alpha, beta, rho) models in the context of financial options. Our network is trained on historical traded option contracts and calculated implied volatility values, and used as the selection criteria for candidate sets that minimize arbitrage opportunities in the SABR model optimization process. We propose an implicit formulation that solves for a fixed-point problem between the observed data and generated surfaces, where implicit layers are combined with a transformer architecture to process sequences with varying lengths: contracts with unique strike prices and expiration dates. The surfaces produced with our scoring network are then compared to a baseline linear interpolation model.**

***Index Terms*—Neural Networks, Implied Volatility, Transformers, Quantitative Methods, Implicit Layers.**

## I. Introduction

### A. Options and Implied Volatility

Option contracts are one of the most important types of derivative contracts in the context of financial markets. A derivative is a contract traded with respect to another asset (also called the underlying asset) and an option contract allows the buyer of the contract to sell or buy the specified asset from the seller at a predefined future date. The pricing equations for these derivatives, such as the Black-Scholes model [3], typically require knowing the time to maturity of the contract, the price of the underlying asset to be traded, the market's current risk-free rate, and the price at which the asset will be executed at maturity, also called strike price [13]. The underlying asset's volatility is also an important parameter when calculating the option's price, but knowledge of its true future value is not possible, so typically a market estimate of the future volatility is used. This estimate, called the implied volatility (*IV* or $\sigma_{iv}$) is obtained by numerically solving for the volatility in the pricing equation using market quoted prices.

Having access to the implied volatility values allows investors to make sales at prices consistent with market expectations, that is, the no-arbitrage price for the contract assuming an efficient market [13]. However, the *IV* values corresponding to all strike prices and maturities are not directly observable, as not all possible contract combinations are traded daily. Ensuring a way to obtain these values with confidence and using only readily available data is a crucial task for quantitative analysts [17]. Usual strategies for estimating these values are predominantly based on linear or spline interpolation algorithms [12] or stochastic differential equation models [15] that use numerical methods to approximate the values from existing listed contracts. Most approaches approximate values for the implied volatility using pairs of time to maturity and strike price as variables. This allows visualizing the chosen model results in the form of a surface, where the **XY** axes are the maturity and strike pairs and the **Z** axis is the *IV*. This surface, called the **implied volatility surface** (IVS), denoted by $\mathcal{S}$ is therefore a function of the maturity time $t$ and the strike price $K$,

$$\mathcal{S} \coloneqq f(t, K), \quad \text{where } f\colon \mathbb{R}^2 \to \mathbb{R}. \tag{1}$$

Ensuring accurate predictions of the IVS is crucial, as even small errors can lead to incorrect pricing and consequently large losses of profit or even negative return [11] given the leveraging capabilities of options when compared to non-derivative securities. Classic models such as linear interpolation, polynomial fitting, and *splines* also lack flexibility for extrapolation and often generate either non-smooth surfaces, arbitrage opportunities, or both [11].

### B. Contributions of the Presented Work

In this paper, we address the challenges of interpolation and extrapolation of the IVS, focusing on reducing arbitrage opportunities. The chosen method uses a stochastic differential equation solution approach, specifically the *Stochastic Alpha, Beta, Rho* (*SABR*) model. The parameters $\alpha$, $\beta$, $\rho$, and $\nu$ of the *SABR* model are obtained through the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton non-linear optimization algorithm [4], [7], [8], [23]. This step is commonly performed using only a subset of the daily observed prices [18], due to the fact that some points may contain unreliable prices of illiquid contracts that do not match the real prices of the derivative, and their usage in the optimization step increases the variance of the final model. To solve this difficulty, the points used as input for the BFGS algorithm are often manually filtered according to criteria set by human experts [9]. Instead of manually selecting these candidate points, models capable of automatically receive point clouds and return a numerical score to be used as selection criteria, such as transformers, have been proposed [2], [5], [14], [18].

The main contribution of this research will be the adaptation of *neural implicit layers*, a physics-inspired variation of fully

connected linear layers where layer weights are optimized to solve for a fixed point equation, to be used together the transformer architecture [25] for the task of filtering candidate points for the *SABR* model This approach is an implicit formulation variation for the scoring model proposed by Kim et al. (2021) [18], aiming to enforce a zero arbitrage condition in the SABR optimization process. This type of layer has been shown to effectively represent smooth surface densities and demonstrated state-of-the-art results in function fitting tasks similar to the one presented in this paper [16], all while being computationally faster for solving interpolation problems than linear layer models with similar performance [20]. Additionally, transformers have also been shown to perform better at handling unordered point clouds over time compared to recurrent neural layers [19] for IVS interpolation tasks, making them a more suitable choice for the task at hand. While directly fitting a neural network to the IVS is possible, optimizing the parameters for a domain-specific model in the context of IVS interpolation has been shown to be more effective [1] due to the necessity of enforcing smoothness and reducing arbitrage opportunities.

### C. The Transformer Architecture

The aforementioned *SABR* model is a parametric stochastic volatility approach for obtaining smooth volatility surfaces and the transformer architecture is used as the scoring model to filter the candidate points for the optimization process. This score, called the *summarized suitability* score (*SS* score), is assigned to each daily traded options contract, and indicates how adequate a candidate point is at summarizing the format of the resulting surface. It measures how much removing a candidate point would affect the generated surface shape and smoothness[1]. The highest scoring points — which best represent the surface — are then selected and used as input for the BFGS algorithm used to fit the *SABR* model parameters. This process aims to remove outlier points that might introduce variance, and consequently arbitrage opportunities [18], in the optimization process for each of the *SABR* models[2].

### D. Fixed-Point problems

Differently from the usual function approximation tasks, a fixed point problem is instead defined by conditions that the output must satisfy. For explicit functions, a given output $z$ is computed directly from the input $x$ using a function $f$, such that $z = f(x)$. In contrast, an implicit function $g$ is such that it depends on both the input $x$ and the expected output $z$, where $z$ is a solution for the equation:

$$g(x, z) = 0 \tag{2}$$

[1]The use of a per-point score is similar to what the principal components of a collection of points are when using the Principal Component Analysis (PCA) technique.

[2]A separate model is fitted for each unique strike price in a given day. Section II-A covers additional details on fitting the *SABR* model at each unique strike price.

Formulating the problem as a fixed point equation can be used for a multitude of problems, including finding fixed points in recurrent networks, solutions to differential equations leading to neural ordinary differential equations, and several physics optimization problems [6]. The implicit formulation offers several practical advantages, primarily the separation of the solution method from the layer definition which can be useful in maintaining interpretability of the resulting model [16] and as a way to enforce constraints in the optimization process, the latter being the main focus of this work.

Moreover, the resulting implicit layer has significant benefits, specifically in deep learning and automatic differentiation frameworks [20], [22]. Automatic differentiation methods store the entire computation graph, which can be memory-intensive. With implicit layers, however, the implicit function theorem allows computing gradients directly at the solution point and therefore storing intermediate variables becomes redundant [22]. This improves memory efficiency, making implicit layers particularly advantageous when substituting existing layers within large deep learning models [21].

### E. Anderson Acceleration for Implicit Layers

The Anderson acceleration (AA) method is a technique used to accelerate the convergence of fixed-point iterations. It is particularly useful when the fixed-point iteration is slow to converge or when the fixed-point iteration is used in a deep learning context. Instead of using the current fixed-point iteration as the next guess, as in standard fixed-point iteration, a linear combination of the previous guesses is used to extrapolate the next, according to the following expression:

$$x_{k+1} = \beta_k x_k + (1 - \beta_k) x_{k-1}$$

where $\beta_k > 0$ is the mixing factor (also called relaxation parameter) used to determine the extrapolation weight. Two additional hyperparameters are used: the damping factor $\lambda$ used to stabilize the fixed-point iteration; and a memory size $m$ that determines the number of previous computed points to use in the extrapolation. Our proposed architecture applies the AA method as shown in Algorithm 1 to solve a fixed-point equation within the implicit layers, and the layer weights are adjusted afterward via backpropagation.

The final deep equilibrium model used within the implicit layer (Algorithm 2) uses the forward pass of a linear layer as input to the Anderson acceleration algorithm, where the solution to a fixed-point equation is computed. This numerical solution can then be used as input for the next layer, or as the final solution to the fixed-point equation itself if the layer is last in the network. When generating the IVS, the output of a transformer encoder is used as input for the implicit layer, and a mean absolute error (MAE) function between the observed surface and SABR-generated surface serves as target function within the fixed point iteration to minimize arbitrage opportunities for listed options in the generated surface.

**Algorithm 1** Equilibrium Model with Anderson Acceleration

**Require:** Function $f$, initial point $x_0$, memory size $m$, damping factor $\lambda$, maximum iterations $K$, tolerance $\epsilon$, mixing factor $\beta$

Initialize $X_0 = x_0, X_1 = F_0 = f(x_0), F_1 = f(F_0)$
**for** $k = 2$ to $K$ **do**
  $j \leftarrow \min(k, m)$
  $G = F_{0,\dots,j} - X_{0,\dots,j}$
  $H = GG^\top + \lambda I$
  Solve $H^{-1} \cdot y = \alpha$
  $\alpha \leftarrow \alpha_{1,\dots,n+1}$
  $X_k \leftarrow \beta\alpha^\top F_0 + (1-\beta)\alpha^\top X_0$
  $F_k \leftarrow f(X_k)$
  Compute residual:

$$\text{residual} = \frac{\|F_k - X_k\|}{\|F_k\| + 10^{-5}}$$

  **if** residual $< \epsilon$ **then**
    **break**
  **end if**
**end for**
**return** $X_k$

**Algorithm 2** Implicit layer forward pass with Anderson acceleration (AA).

**function** IMPLICITLAYER($x, m, \lambda, \epsilon, \beta$)
  **function** $f(x) \leftarrow W \cdot x + b$
  $x \leftarrow \texttt{AA}(f, x, m, \lambda, \epsilon, \beta)$
  $y \leftarrow x \cdot W^T + b$
  **return** $y$
**end function**

## II. Implementation of a Parametric SABR Model and Implicit Neural Network Architectures

### A. Defining the SABR Model Equations

The SABR model is a stochastic differential model, defined by the following equations in which the dynamics of the asset price and the corresponding volatility levels are used to calculate an approximation to the implied volatility where no contract is traded.

- **Alpha** ($\alpha$): This is the initial volatility level.
- **Beta** ($\beta$): The elasticity parameter between the volatility of the asset and its price.
- **Rho** ($\rho$): Statistical correlation between the asset price and its volatility, also called *spot-volatility correlation*.
- **Volvol** ($\nu$): The volatility of the volatility process itself.

The simplified function for the implied volatility using the SABR model and logarithmic "moneyness" $x = \log\left(\frac{K}{F_t}\right)$ is then given by [9]:

$$F_t = Se^{(r-q)t}$$

$$\sigma_{BS}(F_t, K) = \alpha \frac{K^{\frac{1-\beta}{2}}}{1 + \left(\frac{(1-\beta)^2\rho^2}{24} + \frac{(1-\beta)^2(2-3\rho^2)}{1920}\right)(F_tK)^{1-\beta}}$$

where $S$ is the underlying asset price, $r$ is the risk-free rate, $q$ is the dividend yield, $t$ is the time to maturity, $K$ is the strike price and $f(t)$ is the forward price of the asset for a given time to maturity. Refer to Appendix A for the full equation.

Our approach optimizes one SABR model for each unique strike price per day. Three continuous functions map from continuous maturities to the model parameters: $\alpha$, $\rho$, and $\nu$. This ensures a smooth transition between the different strike levels, as using a single SABR model alone can introduce discontinuity in the resulting implied volatility surfaces [10], [18]. Given the functions for the parameters $\alpha_t = f_\alpha(t, \mathbf{P})$, $\rho_t = f_\rho(t, \mathbf{Q})$, and $\nu_t = f_\nu(t, \mathbf{R})$ (the value for $\beta$ is constant and used as hyperparameter), and the encoding variables **P, Q**, and **R** respectively, such that $\mathbf{P} \in \mathbb{R}^5$, $\mathbf{Q} \in \mathbb{R}^4$, and $\mathbf{R} \in \mathbb{R}^4$, the functions for the parameters can then be expressed in terms of the encoding variables and continuous time to maturity $t$ as follows:

$$f_\alpha(t, \mathbf{P}) = P_0 + \frac{p_3}{P_4}\left(\frac{1 - e^{-P_4 t}}{P_4 t}\right) + \frac{P_1}{P_2}e^{-P_2 t}$$

$$f_\rho(t, \mathbf{Q}) = Q_0 + Q_1 t + Q_2 e^{-Q_3 t}$$

$$f_\nu(t, \mathbf{R}) = R_0 + R_1 t^{R_2} e^{R_3 t}$$

Therefore, the task becomes obtaining the encoding variables **P**, **Q**, and **R** for the daily models. We use the Broyden-Fletcher-Goldfarb-Shanno (*BFGS*) algorithm to optimize the encoding variables for minimum mean absolute error (MAE) between the observed implied volatilities and corresponding SABR-generated implied volatilities. The resulting SS scores are then calculated as the MAE between the fitted values for $f_\alpha(t, \mathbf{P})$, $f_\rho(t, \mathbf{Q})$, $f_\nu(t, \mathbf{R})$ and $f_\alpha(t, \widetilde{\mathbf{P}})$, $f_\rho(t, \widetilde{\mathbf{Q}})$, and $f_\nu(t, \widetilde{\mathbf{R}})$, where $\widetilde{\mathbf{P}}$, $\widetilde{\mathbf{Q}}$, and $\widetilde{\mathbf{R}}$ are the encoding variables obtained using a subset of the daily candidate set according to a Monte Carlo estimation detailed in Section II-B3. This process is used to estimate how much each candidate point contributes to the generated volatility surface shape, similarly to what a human trader would manually calculate for each listed contract when looking for arbitrage opportunities [18]. A visual representation of a surface obtained without filtering, and containing an arbitrage opportunity is shown in Figure 1. Finally, a Root Mean Squared Error (RMSE)[3] between the Monte Carlo estimated and transformer predicted SS scores is used as loss function for the neural network model training.

### B. Proposed Architecture

As mentioned, transformers are capable of capturing complex dependencies in sequential non-fixed length data. Within our proposed architecture, we use a transformer encoder to approximate the *SS* scores for each candidate point $\mathcal{S}_i^t \in \mathcal{S}^t$ from the daily observed input.

[3]The RMSE between the SS scores is not to be confused with the MAE between the calculated SABR parameters. The first is used as the loss function when training the neural network model, while the second is used as the BFGS objective function when optimizing for the encoding variables that best replicate the observed daily surface.

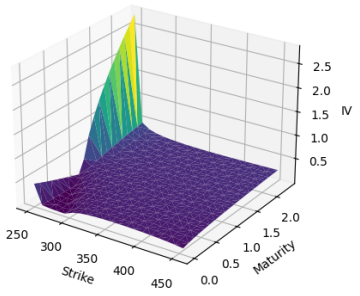


Fig. 1: Surface generated using SABR models fit on entire candidate set, resulting in an outlier peak.

*1) Details on the Chosen Transformer Encoder:* Unlike recurrent neural networks (RNNs) and convolutional neural networks (CNNs), transformers do not rely on a fixed-length data sequence or fixed window size [26]. Instead, they use an attention mechanism, which allows each input element (or token) to consider the relationship with all other elements in the same sequence simultaneously [24]. This is achieved through a block of layers using attention heads.

The attention mechanism accepts continuous data and calculates a value for different heads through the value ($V$), key ($K$), and query ($Q$) matrices, resulting in an output weighted by the importance of each element in the sequence. Specifically, for our problem we use the mechanism of self-attention, and therefore the three matrices are equal.

$$\textit{Attention}(Q, K, V) = \textit{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{3}$$

Transformers are classified as machine learning algorithms due to their ability to learn patterns and relationships from data, and were originally proposed for natural language processing tasks. In the context of our specific architecture, we use only the encoder block and adapt it to perform a regression on the SS scores. The hidden values are passed to fully-connected (FC) layers, and implicit fixed-point iteration layers are added as final output layers.

*2) Details on the Implicit Layer:* Using a Deep Equilibrium Model variation for the Implicit Layer, we adapt Equation 2 by using the following fixed point equation instead, where $g$ is the MAE between the observed surface and the generated surface using the SABR model optimized on the filtered candidate set.

$$\begin{aligned} g_{\mathbf{W}}(x, z^*) &= 0 \\ z^* &= f(z^*, x) \end{aligned} \tag{4}$$

where $\mathbf{W}$ are the layer weights and biases, $f$ is an activation function, and $z*$ is the target output of a batched non-normalized Jacobian to be used as the gradient function in our code, given by the equation below. Finally, $y$ are the hidden values as well as the final output scores.

$$\left(\frac{\partial z^*(\cdot)}{\partial(\cdot)}\right)^T y = \left(\frac{\partial f(z^*, x)}{\partial(\cdot)}\right)^T g_{\mathbf{W}}$$

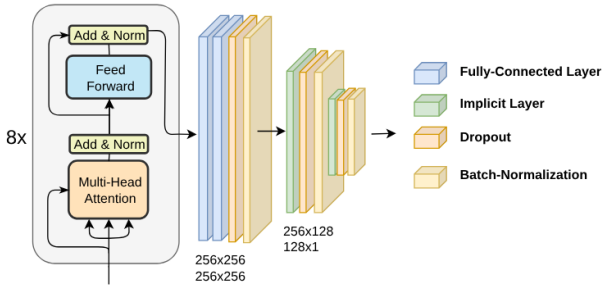


Fig. 2: Visualization of the Transformer Encoder and Implicit Layers used in the proposed architecture.

*3) Monte Carlo Estimation of Z-Scores:* As mentioned in Section II-A, a Monte Carlo process is used to estimate synthetic target SS scores for the model. Instead of using the entire daily candidate set for calculating the encoded variables **P**, **Q**, and **R** for the SABR model, only $k$ small subsets with size $p \cdot n$, where $n$ is the total number of candidate points and $p$ is a percentage value, of the candidate set are used as input to the *BFGS* algorithm to reduce surface volatility, resulting in $\widetilde{\mathbf{P}}$, $\widetilde{\mathbf{Q}}$, and $\widetilde{\mathbf{R}}$. The SS score is then calculated as the mean absolute error (MAE) between the functions $f_\alpha$, $f_\rho$, and $f_\nu$ using both encoded variable types as shown in Algorithm 3 and its normalized values are used as target values for our model. The choice of using a neural network approach for calculating the SS scores is motivated by the much higher computational cost of the Monte Carlo estimation, as well as the possibility of using previous observations to calculate the current day's scores.

This estimation using a finite number of subsets is necessary due to computational constraints, both in terms of time and memory. Finally, model training is performed until convergence to low RMSE values between predicted scores and estimated scores. After training, the model prediction substitutes the Monte Carlo estimation for the SS scores, and the best candidate set is selected and passed to the *BFGS* algorithm to obtain the final encoding variables for the SABR models when generating the daily surface.

## III. Methodology and Results

### A. Data Description and Pre-Processing

The dataset used for training was obtained from historical traded option contracts on the *SPY ETF*[4] from January 2021 to December 2022, with a total of 68 unique days. The dataset contains various columns on traded options, indexed by date, including strike prices, future contract prices, maturities,

[4]An exchanged traded fund replicating the S&P500 index

**Algorithm 3** Monte Carlo Estimation of Target Z-Scores

**Require:** Daily candidate set $\mathcal{S}^t$, percentage $p = 0.8$, number of subsets $k = 1 \times 10^2$

$\alpha^*, \rho^*, \nu^* \leftarrow BFGS(\mathcal{S}^t)$
$\text{SS}_\alpha, \text{SS}_\rho, \text{SS}_\nu \leftarrow \mathbf{0}$
**for** candidate $c$ in $\mathcal{S}^t$ **do**
  $\mathcal{S}_i^t \leftarrow \text{Sample}(\mathcal{S}^t, k, p)$
  $\widetilde{\mathbf{P}}, \widetilde{\mathbf{Q}}, \widetilde{\mathbf{R}} \leftarrow BFGS(\mathcal{S}_i^t)$
  $\text{SS}_{\alpha,c} \leftarrow \text{MAE}(f_\alpha(t, \widetilde{\mathbf{P}}) - \alpha^*)$
  $\text{SS}_{\rho,c} \leftarrow \text{MAE}(f_\rho(t, \widetilde{\mathbf{Q}}) - \rho^*)$
  $\text{SS}_{\nu,c} \leftarrow \text{MAE}(f_\nu(t, \widetilde{\mathbf{R}}) - \nu^*)$
**end for**
z-normalize $\text{SS}_\alpha, \text{SS}_\rho, \text{SS}_\nu$

implied volatilities, among others. Relevant to our study, the principal columns used were:

- **Underlying**: Price of the underlying asset.
- **Strike**: Option's strike price.
- **Expiration**: Days remaining until the option's expiration.
- **Implied Volatility**: Value for the implied volatility.
- **Date**: Traded date for the contract, used as the preprocessed dataset index.

Additionally, two columns were added to be used for the Monte Carlo estimation:

- **Maturity**: Calculated by dividing *Expiration* by 252 (working days in a year) to normalize the maturity time.
- **Risk-free rate**: Risk-free rate obtained from the Federal Reserve Economic Data (FRED) for the corresponding period.
- **Dividend**: Dividend yield paid quarterly by the SPY index.

### B. Data Preprocessing

The data preprocessing involved several essential steps, predominantly to allow the use of the surface data directly in the transformer architecture and subsequent generation of the surface for historical data. Each of the trained transformers (one for each parameter $\alpha, \rho$, or $\nu$) takes four dimensions as input, which are:

1) Time to maturity;
2) The daily parameters $\alpha, \rho$, or $\nu$ calculated using the *BFGS* optimization algorithm;
3) **1** if the parameter was calculated for the current day, **0** if it was calculated for a fixed set of maturities (top 7 most frequent maturities) using the previous day fitted parameters;
4) Parameter value calculated using the previous day's $\mathbf{P}$, $\mathbf{Q}$ and $\mathbf{R}$ values for same strike and maturity.

The target scores were generated before training using the Monte Carlo algorithm, detailed in Section II-B3. with $p = 80\%$ and $k = 1 \times 10^2$. For training and testing we also observed that the choice of $\beta = 0.5$ performed best at minimizing the surface's MAE.

### C. Results and Surface Comparison

A baseline linear interpolation for the daily observed points was also used to compare our trained model's surfaces by interpolating the points within a fixed grid of strikes and maturity. No extrapolation was performed due to the nature of the linear model.

The training dataset had a total of 44817 observations totaling 68 unique days, which lead to the decision of using a leave-one-out cross-validation approach. The Adam algorithm was used as optimizer, with a max epoch range of $10^3$ and an early stopping sensitivity of $10^{-3}$. The Anderson acceleration was implemented with a maximum iteration limit of 10, a convergence threshold of $\epsilon = 10^{-4}$, damping factor of $\lambda = 0.1$, and memory size of $m = 5$. A batch size of 8 was used to avoid memory issues when training the implicit layer. The network architecture is a transformer encoder block with 8 sequential attention heads, 2 linear layers with 256 hidden units and 2 implicit layers with 256 and 128 hidden units, respectively, and a dropout rate of 0.1 was used for the dropout layers. An illustration of the architecture is shown in Figure 1.

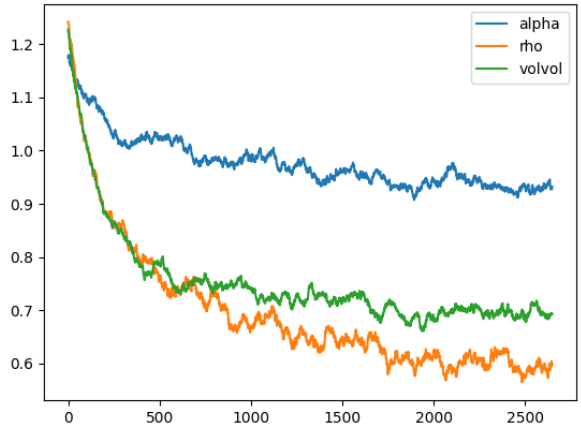


Fig. 3: Historical loss curve on training session data. Curves were cut for better visualization, with the largest episode being 2950 epochs for the $SST_\alpha$ (alpha) network.

| Model | Test loss (RMSE) | Train time | Preprocess time (Monte Carlo Estimation) |
|---|---|---|---|
| $SST_\alpha$ | 1.21 | 4m21s | 7m50s |
| $SST_\rho$ | 0.72 | 2m8s | 6m34s |
| $SST_\nu$ | 0.75 | 1m43s | 6m41s |
| Total | - | 8m12s | 21m5s |

TABLE I: Model training metrics for each of the three SST models.

For benchmarking, the execution time was measured using the Python `perf_counter` method and a heat-up of the model was performed beforehand on the entire dataset.

Separate tests using a linear interpolation with the original points and with the implicit layer were performed to compare both generated surfaces, shown in Appendix B. The linear interpolation obtained a MAE of $0.0$ to the observed surface, as expected, and the proposed model had a MAE of $7 \times 10^{-4}$ for the points used in the Monte Carlo estimation. While still near the threshold hyperparameter of $1 \times 10^{-4}$, the threshold could be ensured by using a larger maximum iteration limit for the Anderson acceleration, trading off for a larger training and execution time.

The mean train and test losses using LOO cross-validation are shown in Table I, while the execution times shown below in Table II are on the entire dataset. The RMSE values for the test data show that the model was able to generalize well to the test data, with the lowest RMSE value for the $\rho$ parameter.

| Metric | Mean time | Standard Deviation |
|---|---|---|
| Execution time on entire dataset | 229.1ms | 41.2ms |
| Execution time per point | 2ms | 1ms |

TABLE II: Model performance

## IV. Conclusion

### A. Summary

This work presents an architecture drawing on approaches by Kim et al. (2021) [18] and Kolter et al. (2020) [20] to optimize the parameters of multiple SABR models using a neural network with implicit deep equilibrium layers as the candidate set selection mechanism. Our approach was successful in enforcing the no-arbitrage conditions in the resulting surfaces by solving for a fixed-point problem between the observed data and generated IV values. The time complexity of the trained model vastly outperformed the Monte Carlo method, with a speedup of 6300x, and was able to generalize well to test data, as shown in Table II and Table I, respectively. Additionally, the neural network approach supports fine-tuning to incorporate new daily observations; a significant advantage over the Monte Carlo process, which would require estimation from scratch using the new dataset.

### B. Implications and Future Work

The main focus of this work was to merge the approach using the transformer and the implicit layers to ensure no-arbitrage conditions in the SABR model by solving for a fixed-point problem. The approach of using neural networks, even though significantly faster than Monte Carlo methods, could be further optimized for the context of high-frequency intraday trading. Adapting the model to accept exogenous market factors besides the observed data, such as macroeconomic indicators, market indexes, and other financial instruments, could allow the model to better adapt to changing market regimes and is also a promising extension in future research.

## Appendix A
## Complete Implied Volatility Equation for the SABR model

The SABR model is a stochastic volatility model developed as an extension of the Black-Scholes model that allows for the volatility of the asset to be stochastic, and is widely used in the financial industry for option pricing and risk management. The implied volatility of the asset according to the model is given by the following equations [9]:

$$F_t = S \cdot e^{(r-d)t}$$

$$x(t) = \log\left(\frac{F_t}{K}\right)$$

where $F_t$ is the price of a forward contract with the same underlying asset as the option, $S$ is the spot price of the asset, $r$ the risk-free rate, and $d$ the dividend yield. $x(t)$ is the log-moneyness value for the option.

$$y(t) = (F_t K)^{\frac{1-\beta}{2}}$$

$$z(t) = \frac{\nu}{\alpha} x(t) y(t)$$

$$\kappa(t) = \log\left(\frac{\sqrt{1 - 2\rho z(t) + z(t)^2} + z(t) - \rho}{1 - \rho}\right)$$

where $y$ and $z$ are auxiliary functions to simplify the SABR model equation, and $\kappa$ represents the adjustment factor for the volatility.

$$\sigma(t, K) = \alpha t \frac{\left(1 + \left(\frac{(1-\beta)^2\alpha^2}{24y(t)^2} + \frac{\alpha\beta\rho\nu}{4y(t)} + \frac{(2-3\rho^2)\nu^2}{24}\right)\right)}{\left(1 + \frac{(1-\beta)^2 x(t)^2}{24} + \frac{(1-\beta)^4 x(t)^4}{1920}\right)} \frac{z(t)}{y(t)\kappa(t)}.$$

Optimization for the $\alpha, \beta, \rho, \nu$ parameters is usually performed using constrained optimization methods, and other algorithms such as adaptations of the Newton-Raphson method or Sequential Quadratic Programming can be used, but choosing the best performing optimization method is outside the scope of this work.

## APPENDIX B
## COMPARISON OF INTERPOLATED SURFACES

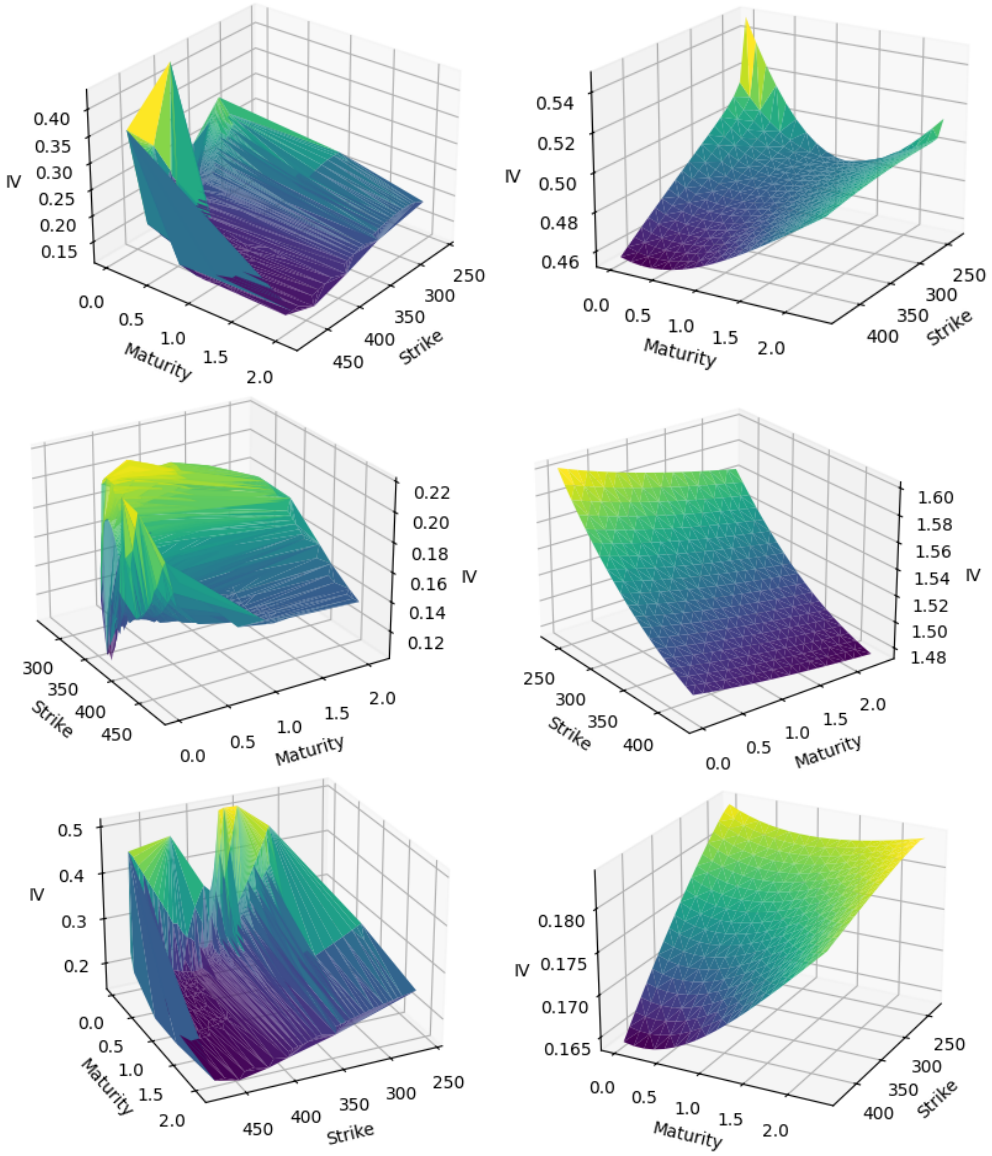


Fig. 4: Surfaces interpolated on test data using a linear interpolation model (left) compared to the SABR models fitted with the candidate set selected using the trained implicit transformer architecture (right).

## REFERENCES


[1] Caio Almeida, Jianqing Fan, Gustavo Freire, and Francesca Tang. Can a machine correct option pricing models? *JOURNAL OF BUSINESS & ECONOMIC STATISTICS*, 41(3):995–1009, JUL 3 2023.

[2] Hyeong-Ohk Bae, Seunggu Kang, and Muhyun Lee. Option pricing and local volatility surface by physics-informed neural network. *COMPUTATIONAL ECONOMICS*, 2024 FEB 17 2024.

[3] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[4] Charles G Broyden. The convergence of a class of double-rank minimization algorithms 1. general considerations. *IMA Journal of Applied Mathematics*, 6(1):76–90, 1970.

[5] Jay Cao, Jacky Chen, and John Hull. A neural network approach to understanding implied volatility movements. *QUANTITATIVE FINANCE*, 20(9):1405–1413, SEP 1 2020.

[6] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. 2019.

[7] Roger Fletcher. A new approach to variable metric algorithms. *Computer Journal*, 13(3):317–322, 1970.

[8] Donald Goldfarb. A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109):23–26, 1970.

[9] Patrick Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Managing smile risk. *Wilmott Magazine*, 1:84–108, 01 2002.

[10] Patrick Hagan, Andrew Lesniewski, and Diana Woodward. *Probability Distribution in the SABR Model of Stochastic Volatility*, volume 110, pages 1–35. 06 2015.

[11] Patrick S. Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Arbitrage-free sabr. *Wilmott*, 2014(69):60–75, 2014.

[12] Cristian Homescu. Implied volatility surface: Construction methodologies and characteristics, 2011.

[13] John C. Hull. *Options, Futures, and Other Derivatives*. Pearson Education Limited, 10th edition, 2018.

[14] Yacin Jerbi and Samira Chaabene. European call price modelling using neural networks in considering volatility as stochastic with comparison

to the heston model. *Journal of Statistical Computation and Simulation*, 90(10):1793–1810, 2020.

[15] Richard Jordan and Charles Tier. Asymptotic approximations to cev and sabr models. *SSRN Electronic Journal*, pages 1–38, May 2011. Available at SSRN: https://ssrn.com/abstract=1850709 or http://dx.doi.org/10.2139/ssrn.1850709.

[16] Kenji Kawaguchi. On the theory of implicit deep learning: Global convergence with implicit layers. 2021.

[17] Bo-Hyun Kim, Daewon Lee, and Jaewook Lee. Local volatility function approximation using reconstructed radial basis function networks. In J Wang, Z Yi, JM Zurada, BL Lu, and H Yin, editors, *ADVANCES IN NEURAL NETWORKS - ISNN 2006, PT 3, PROCEEDINGS*, volume 3973 of *Lecture Notes in Computer Science*, pages 524–530. SPRINGER-VERLAG BERLIN, 2006. 3rd International Symposium on Neural Networks (ISNN 2006), Chengdu, PEOPLES R CHINA, MAY 28-31, 2006.

[18] Hyeonuk Kim, Kyunghyun Park, Junkee Jeon, Changhoon Song, Jungwoo Bae, Yongsik Kim, and Myungjoo Kang. Candidate point selection using a self-attention mechanism for generating a smooth volatility surface under the sabr model. *EXPERT SYSTEMS WITH APPLICATIONS*, 173, JUL 1 2021.

[19] Soohan Kim, Seok-Bae Yun, Hyeong-Ohk Bae, Muhyun Lee, and Youngjoon Hong. Physics-informed convolutional transformer for predicting volatility surface. *QUANTITATIVE FINANCE*, 24(2):203–220, JAN 9 2024.

[20] Zico Kolter, David Duvenaud, and Matt Johnson. Deep implicit layers - neural odes, deep equilibrium models, and beyond, 2020. NeurIPS 2020 tutorial. Retrieved from https://implicit-layers-tutorial.org/.

[21] Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations. 2020.

[22] Stefano Massaroli, Michael Poli, Jinkyoo Park, Atsushi Yamashita, and Hajime Asama. Dissecting neural odes. 2021.

[23] David F Shanno. Conditioning of quasi-newton methods for function minimization. *Mathematics of Computation*, 24(111):647–656, 1970.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2023.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Ilya Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pages 5998–6008. Curran Associates Inc., 2017.

[26] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. 2023.